

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..10

TROUBLESHOOTING

 Copy as Markdown

Troubleshoot Google sign-in

This page helps you identify common Sign in with Google failures, explains why they occur, and shows how to resolve them. For platform-specific issues with the underlying Flutter package, see the [google_sign_in_android troubleshooting guide](#).

Setup checklist

Go through this before investigating a specific error. Most problems come from a missed step.

Google Cloud

- ☐ Create a **Google Cloud project** in the [Google Cloud Console](#).
- ☐ In **Google Auth Platform**, complete the initial setup (wizard) and add the required scopes on **Data Access** (`.../auth/userinfo.email` and `.../auth/userinfo.profile`).
- ☐ On **Branding** ([Branding](#)), complete the OAuth consent screen (logo, homepage, privacy policy, terms of service, and developer contact) and add the **root domain** (top private domain) under **Authorized domains**. Google stores only the root, so a single verified entry covers all of its subdomains. On Serverpod Cloud, add `serverpod.space` (already verified by Serverpod, no DNS setup needed). For custom domains, see [Verify your authorized domain](#).
- ☐ Add **test users** on **Audience** while in **Testing** mode ([Audience](#)), or **Publish app** when everyone should be able to sign in.
- ☐ Create a **Web application** OAuth client. For web sign-in, set **Authorized JavaScript origins** to your Flutter web app's origin (for example, `https://my-awesome-project.serverpod.space`) and **Authorized redirect URIs** to the route URL from [Web setup](#) (for example, `https://my-awesome-project.serverpod.space/auth/callback`), or the `auth.html` URL if you use the [separately-hosted Flutter web](#) fallback (for example, `http://localhost:49660/auth.html`). Copy the **Client ID** and **Client secret**.



☐ Add `googleClientSecret` to `config/passwords.yaml` with your client ID, client secret, and matching `redirect_uris` (the same callback URL as above). For production, this is the route URL you registered via `FlutterWebAuth2CallbackRoute` (for example, `https://my-awesome-project.serverpod.space/auth/callback`) from [Web setup](#), or the production `auth.html` URL on your Flutter web host if you use the [separately-hosted Flutter web](#) fallback. See [Publishing to production](#).

Server

☐ For new or customized servers, confirm auth services and JWT are configured per [Authentication setup](#) before adding Google.

- ☐ Add `GoogleIdpConfigFromPasswords()` to `identityProviderBuilders` in `server.dart`.
- ☐ Create a `GoogleIdpEndpoint` file in `lib/src/auth/`.
- ☐ Start the server with `serverpod start`, then create and apply the migration (press **M**).

Client

☐ Add `client.auth.initializeGoogleSignIn()` after `client.auth.initialize()` inside `initializeClient()` in your Flutter app's `lib/client.dart`. On web, pass `clientId` and `redirectUri` (the full callback URL, either the route URL or the `auth.html` URL, depending on your [Web setup](#)). On Android, pass `serverClientId` (the Web client's ID) unless your app uses the Firebase Gradle plugin.

☐ Surface Google sign-in in the UI with `SignInWidget` or `GoogleSignInWidget` (see [Present the authentication UI](#)).

☐ Create an **iOS** OAuth client in the **same** Google Cloud project as the Web client, using the same **Bundle ID** as the app. Set `GIDClientID` from the iOS client, `GIDServerClientID` to the **Web** client's ID, and add the reversed-client-ID **URL scheme** in `Info.plist` (*iOS only*).

☐ Create an **Android** OAuth client in the **same** project, with the same **package name** and **SHA-1** as the build you run (*Android only*).

- ☐ Set up the web callback (*web only*). Pick one:
 - **Standard:** Register `FlutterWebAuth2CallbackRoute` on `pod.webServer` in `server.dart` before `pod.start()` per [Web setup](#).
 - **Separately-hosted fallback:** Create `web/auth.html` in your Flutter project as described in [Web callback page](#) (`auth.html`) and run Flutter on a fixed `--web-port` so the origin does not change every run. See [separately-hosted Flutter web](#).

Sign-in fails with `redirect_uri_mismatch`

Problem: The OAuth flow fails with a `redirect_uri_mismatch` error from Google.

Cause: The redirect URI sent during sign-in does not exactly match one of the URIs registered on the Web OAuth client. Google requires an exact match (scheme, host, port, path, and casing).

Resolution: In the Google Auth Platform, navigate to **Clients**, select your Web application client, and verify that the URIs under **Authorized JavaScript origins** and **Authorized redirect URIs** match what your app actually uses:

- **Authorized JavaScript origins** must contain your Flutter web app's origin (for example, `http://localhost:49660` locally, or `https://my-awesome-project.serverpod.space` in production).
- **Authorized redirect URIs** must contain the full callback URL: the route URL from [Web setup](#) (for example, `https://my-awesome-project.serverpod.space/auth/callback`), or the full `auth.html` URL if you use the [separately-hosted Flutter web](#) fallback (for example, `http://localhost:49660/auth.html` locally).

The same callback URL must also appear in `client.auth.initializeGoogleSignIn(..., redirectUri: ...)` in your Flutter app.

The `redirect_uris` key in `config/passwords.yaml` must be present for the JSON to parse, but its contents are not used for this check.

Common mistakes:

- Trailing slashes, port differences, or `http` vs `https`.
- Forgetting the callback path on the redirect URI. The bare origin is not enough.
- For separately-hosted Flutter web, the Flutter dev server running on a random port. Pass `--web-port=<port>` to `flutter run` so the origin is stable.
- A stale build on the standard [Web setup](#) flow. The app Serverpod serves is a compiled snapshot, so a `redirectUri` change in `client.dart` takes effect only after re-running `flutter build web`. Rebuild and hard-reload the browser, because the service worker can cache the old bundle.

Production redirect URIs rejected by Google

Problem: When adding your production domain to **Authorized redirect URIs** on the Web OAuth client, Google rejects it with an error about unauthorized domains.

Cause: The redirect URI's root domain (the top private domain) has not been verified and added to **Authorized domains** on the Branding page. Google requires the root to be a verified Authorized Domain before it accepts redirect URIs that use it.

Resolution: Add the root domain (for example, `serverpod.space` or your custom root) to **Authorized domains**, then retry the redirect URI. On Serverpod Cloud, the `serverpod.space` root is already verified, so you only need to add `serverpod.space` to **Authorized domains**. For a custom domain, verify ownership at [Google Search Console](#) first. See [Verify your authorized domain](#).

Sign-in works for you but not for other users

Problem: Sign-in works for your Google account but other users get an error screen from Google saying the app is not verified or access is denied.

Cause: Your Google Auth Platform app is still in **Testing** mode. Only users explicitly added as test users can sign in (up to 100).

Resolution: Navigate to the [Audience](#) page and click **Publish App** to allow any Google account to sign in. If your app uses sensitive or restricted scopes, Google may require a verification review before publishing.

Sign-in callback fails locally with `flutter run -d chrome`

Problem: You followed [Web setup](#) and registered `FlutterWebAuth2CallbackRoute`. Sign-in completes at Google, the browser redirects, but the Flutter app never receives the result. Only affects `flutter run -d chrome` local dev.

Cause: The integrated route requires Serverpod and your Flutter web app to be on the **same origin** (same scheme, host, AND port). With `flutter run -d chrome`, Flutter runs on its own dev server port (for example, `49660`) while Serverpod is on `8082`, so they are different origins. The browser blocks the callback page's `postMessage` across origins.

Resolution: Use the [separately-hosted Flutter web](#) flow for local dev. It serves `auth.html` from Flutter's own dev server, same-origin with your Flutter app. For production, the integrated route works once Serverpod serves your Flutter build (via `FlutterRoute`, mounted at `/` on default projects, or `/app` when the website option is enabled).

Sign-in callback never returns to the Flutter app

Problem: The Google sign-in window completes successfully, but the Flutter app never receives the result. The browser sits on a blank page, or `signIn()` hangs.

Cause: The browser was redirected to a URL that does not serve the callback page, or the page is loaded but cannot post the result back to the Flutter app (origin mismatch).

Resolution:

1. Confirm your callback page is reachable. Open the `redirectUri` directly in a browser tab. You should see the "Authentication complete" page.
 - For the standard [Web setup](#), confirm `FlutterWebAuth2CallbackRoute` is registered on `pod.webServer` before `pod.start()` and that the path matches the URL you opened.

- For the [separately-hosted Flutter web](#) fallback, confirm `web/auth.html` exists in your Flutter project and contains the script described in [Web callback page \(auth.html\)](#). If the file is missing, the redirect URL returns a 404.
2. Confirm the `redirectUri` passed to `initializeGoogleSignIn` exactly matches the URL where the callback is served, and that both share scheme, host, and port with your Flutter web app (the browser blocks `postMessage` across origins).

Server fails to parse googleClientSecret from passwords.yaml

Problem: The server crashes on startup with a JSON parsing error related to `googleClientSecret`.

Cause: The YAML block scalar indentation is incorrect. The `googleClientSecret` key uses `|` (literal block scalar), which requires every line of the JSON to be indented at the same level relative to the key.

Resolution: Make sure the JSON block is indented consistently under the `|`:

```
development:
  googleClientSecret: |
    {
      "web": {
        "client_id": "...",
        "client_secret": "...",
        "redirect_uris": []
      }
    }
```

Every line of the JSON must be indented by at least one level more than `googleClientSecret:`. Mixing tabs and spaces can also cause issues.

Sign-in fails on Android with PlatformException(sign_in_failed) or clientConfigurationError

Problem: Google sign-in throws a `PlatformException(sign_in_failed, ...)` or a `GoogleSignInException` with `clientConfigurationError` on Android but works on other platforms.

Cause: The SHA-1 fingerprint registered in your Android OAuth client does not match the signing key used to build the app. This commonly happens when switching between debug and release builds, or when the app is signed with a different keystore than the one registered.

Resolution:

1. Check which SHA-1 your debug build is using:
-

```
./gradlew signingReport
```

2. In the Google Auth Platform, navigate to **Clients** and verify your Android OAuth client has the correct SHA-1 fingerprint.

3. If you are testing a release build, use the SHA-1 from your production keystore:

```
keytool -list -v -keystore /path/to/keystore
```

4. After updating the SHA-1, it can take a few minutes for Google to propagate the change.

Sign-in works in debug but fails in release

Problem: Google sign-in works in debug mode but fails silently or with `sign_in_failed` in a release build.

Cause: Debug and release builds use different signing keys. The SHA-1 fingerprint registered in your Android OAuth client only matches the debug keystore.

Resolution: Register the SHA-1 fingerprint from your release keystore as an additional fingerprint in the Google Auth Platform. You can add multiple SHA-1 fingerprints to the same Android OAuth client, or create separate clients for debug and release.

Sign-in fails on Android with "serverClientId must be provided"

Problem: Sign-in fails on Android with `GoogleSignInException(code GoogleSignInExceptionCode.clientConfigurationError, serverClientId must be provided on Android, null)`.

Cause: On Android, the `google_sign_in` SDK requires the server (Web application) client ID, and nothing supplies it. A plain Flutter project does not read `google-services.json`: that file is consumed by the `com.google.gms.google-services` Gradle plugin, which only Firebase-based projects apply.

Resolution: Pass the Web application client ID when initializing, as shown in [Initialize the Google sign-in service](#):

```
client.auth.initializeGoogleSignIn(  
  serverClientId: '<web_client_id>.apps.googleusercontent.com',  
);
```

You can also supply it at build time with `--dart-define=GOOGLE_SERVER_CLIENT_ID=...`. See [Configuring client IDs on the app](#).

For Firebase-based projects using the Gradle plugin, make sure a Web application OAuth client exists in the same Google Cloud project and re-download `google-services.json` so it includes the web client entry.

Endpoint calls fail on Android with connection refused

Problem: Sign-in completes at Google, but the app then fails with

```
ServerpodClientNetworkException: ... Connection refused ...  
uri=http://localhost:8080/...
```

Cause: On Android, `localhost` is the emulator or device itself, not the machine running your server. The project template's `assets/config.json` sets `apiUrl` to `http://localhost:8080`, and that value takes precedence over the framework's platform-aware default (see [server URL resolution](#)).

Resolution: Point the app at your server explicitly when running on Android:

```
flutter run --dart-define=SERVER_URL=http://10.0.2.2:8080/
```

On the Android emulator, `10.0.2.2` maps to the host machine. On a physical device, use your computer's LAN IP address instead (for example, `http://192.168.1.20:8080/`), with the phone on the same network.

Server crashes on first Google sign-in with "no such table"

Problem: The server builds and starts, but crashes when a user tries Google sign-in. The error cites a missing table (like `serverpod_auth_idp_google_account`).

Cause: The database migration that creates the provider's tables was never created or applied.

Resolution: In the running `serverpod start` terminal, press **M** to create and apply the migration.

Google sign-in button does not appear

Problem: The `SignInWidget` renders, but the Google button is missing.

Cause: The `SignInWidget` shows the Google button when the client has a registered `GoogleIdpEndpoint`. On web, the Google sign-in service must also be initialized. The common misses:

- `GoogleIdpEndpoint` is missing on the server, or the client was not regenerated after adding it.

- On web, the running app does not include your `initializeGoogleSignIn` call yet. The app Serverpod serves is a build, and hot reload does not re-run `initializeClient()`.
- On web, `initializeGoogleSignIn` threw an `ArgumentError` because `clientId` or `redirectUri` is missing. See [clientId or redirectUri is required on web](#).

Resolution: Confirm `GoogleIdpEndpoint` exists on the server and run `serverpod generate`. On web, run the `flutter build web` command from [Web setup](#) again and hard-reload the browser. If you run the app with `flutter run` instead, hot restart it.

Lightweight sign-in (One Tap) not appearing

Problem: You enabled `attemptLightweightSignIn: true` but the silent sign-in doesn't trigger.

Cause: On web, `attemptLightweightSignIn` has no effect at all in this version, so nothing appearing there is expected. On Android and iOS, lightweight sign-in requires that the user has signed in with Google on the device before and has a Google account configured.

Resolution: On mobile, this is expected behavior for first-time users, since the prompt only appears for returning users, and Google may suppress it temporarily after repeated dismissals. The regular sign-in button remains available as a fallback.

iOS sign-in prompt doesn't show

Problem: Tapping the Google sign-in button on iOS has no effect or throws an error about a missing client ID.

Cause: The `GIDClientID` or `GIDServerClientID` keys are missing or incorrect in `Info.plist`, or the URL scheme is not registered.

Resolution:

1. Open `ios/Runner/Info.plist` and verify that `GIDClientID` is set to the `CLIENT_ID` from your iOS OAuth client plist, and `GIDServerClientID` is set to the client ID from your Web application OAuth client.
2. Verify the URL scheme (`CFBundleURLSchemes`) contains the reversed client ID from the iOS plist (the `REVERSED_CLIENT_ID` value).
3. Clean the build and run again.

clientId or redirectUri is required when initializing Google Sign-In on web

Problem: The Flutter app throws an `ArgumentError` at startup with one of these messages:

- `redirectUri is required when initializing Google Sign-In on web`

- `clientId` is required when initializing Google Sign-In on web with a redirect URI

Cause: On web, `initializeGoogleSignIn` needs both `clientId` and `redirectUri`. If you omit `clientId`, it reads the value from the `GOOGLE_CLIENT_ID` `--dart-define`. There is no such fallback for `redirectUri`.

Resolution: Pass both values on web:

```
client.auth.initializeGoogleSignIn(  
  clientId: kIsWeb  
    ? 'your-web-client-id.apps.googleusercontent.com'  
    : null,  
  redirectUri: kIsWeb  
    ? 'https://my-awesome-project.serverpod.space/auth/callback'  
    : null,  
);
```

You can also pass `clientId` via `--dart-define` instead of in code:

```
flutter run --dart-define=GOOGLE_CLIENT_ID=your-web-client-id.apps.googleusercontent.com  
...
```

Google API calls fail after one hour

Problem: Your server calls Google APIs (for example, Calendar or Drive) with the access token captured during sign-in, but requests start returning `401 Unauthorized` after about an hour.

Cause: Google access tokens expire after 3,600 seconds (one hour). Serverpod captures the token during sign-in for the `getExtraGoogleInfoCallback`, and does not refresh it afterwards.

Resolution: Fetch what you need inside `getExtraGoogleInfoCallback` while the token is fresh. For ongoing access, ask the user to sign in again, or run your own token exchange in a custom endpoint so you control the refresh token.

Related

- [Setup](#): configure the Google Auth Platform and register the identity provider.
- [Customizations](#): configuration options and sign-in UI customization.
- [UI components](#): the sign-in widgets and how to compose them.